Nexora School Exam Platform

Final MVP Implementation Documentation

Version 3.0 | 12 October 2026 | Target delivery Saturday 17 October 2026

Build a working cloud application that takes a school from question upload to an approved exam, synchronized student sitting, released results and repeat practice. This guide fixes the MVP scope, data contracts, failure behavior and daily delivery checkpoints. It replaces the previous implementation timeline with a focused build for this week.

Delivery target: a deployed, integrated MVP with the complete objective-exam workflow. **Team assumption:** two focused implementation tracks, existing frontend scaffolding where usable, and daily owner acceptance. A 500-student sitting is a measured release target, not a subscription limit or an already-proven capacity.

Contents

Section	Page
1. About this document	2
2. Product overview and MVP scope	3
3. System architecture	4
4. Technology stack and version policy	5
5. Frontend architecture and components	6
6. Frontend feature catalogue	7
7. Question import and AI preparation	8
8. Exam engine and submission protocol	10
9. Backend architecture and rules	12
10. Data model and integrity	13
11. Security and school isolation	15
12. API contracts	16
13. Delivery plan to Saturday	18
14. Deployment and operations	20
15. Risks and release decisions	21
16. Working with coding assistants	22
17. Roadmap after the MVP	23
18. Worked examination and glossary	24
19. Decisions and technical references	25

All delivery times use Africa/Addis_Ababa, EAT, UTC+3. Saturday means 17 October 2026.

1. About this document

1.1 Purpose and authority

This is the build reference for Nexora's frontend developer, backend developer and reviewer. Implement the rules here in the API, database and browser, then verify the complete workflow on the deployed application. Product decisions come from Nexora; repository evidence determines what is already built.

1.2 Status labels

Label	Meaning
Required	A product behavior that the MVP must provide.
MVP decision	The implementation default for this week; change it deliberately in the decision log.
Reported	Stated in the supplied PDF, but not independently checked against the repository.
Verify Monday	A concrete dependency to check before relying on the delivery plan.

1.3 Corrections to the supplied implementation

Earlier assumption	Final direction
Phones deferred; desktop exam layout	Student login, exam runner, results and practice support phones from the first build.
Teacher publishes and releases results	Teacher submits content. School admin approves, schedules, releases results and enables practice.
Personal start timer after clicking Start	Preparation time is chosen by admin; common start is exactly ten minutes later.
Imports, charts and exports are backlog	Template DOCX/XLSX import, AI enrichment, basic charts and result exports are in this MVP.
Email required for each student	Generate a unique user ID and password; student email is optional and unnecessary for login.
Failed final save can still look submitted	Only a database-committed receipt means Submitted. Pending uploads and late recovery remain explicit.
Hosting unknown; Docker assumed	HahuCloud is the selected starting host. Use its supported Node deployment; do not require Docker.
Nine weeks on mocks, integration later	Integrate with the real API on Tuesday; Friday is reserved for validation and fixes.

1.4 Evidence to retain

The supplied PDF reports a frontend scaffold, shared components and passing CI. Verify these against the actual checkout on Monday; do not copy its exact package versions or "Verified" labels as current facts. Keep useful code, the feature-folder layout, reducer design, typed API errors, IndexedDB queue and shared components.

2. Product overview and MVP scope

2.1 The product

One SaaS codebase serves separate school workspaces. Teachers upload questions and answers. An API model proposes topics and explanations. After staff review, the school administrator approves and schedules the exam. Students use generated credentials on their phones or school computers, receive official results at the chosen time, and practise released papers later.

2.2 Roles and permissions

Role	MVP responsibilities
Nexora operator	Create a school and first administrator through a protected command; operate hosting and support. A platform sales/billing portal is deferred.
School administrator	Manage accounts and classes; approve exams; set preparation time, result date and practice access; review late uploads; view all school results, ranks, analysis and exports.
Teacher	Upload questions and keys, enter exam parameters, review AI output, submit for approval, view and export results for assigned classes and subjects.
Student	Sign in with generated credentials; prepare and sit assigned exams; see released official results; practise enabled exams and see the best practice score.

Invigilation is performed by school staff using the admin exam view. A separate invigilator portal, video monitoring and a live camera grid are outside this week's scope.

2.3 Required by Saturday

- School separation, staff access, CSV roster upload, bulk generated student credentials and administrator password reset.
- Structured DOCX and XLSX imports; single-answer MCQ and true/false; teacher metadata; AI labels and explanations; reviewed admin approval.
- Ten-minute preparation, synchronized start, encrypted preload, on-device answer saving, compact optional cloud saves, final receipt and late-upload review.
- Deterministic objective marking, scheduled results, student marks and competition rank, score distribution and topic-performance charts.
- Admin-enabled repeated practice with correctness and approved explanations; maximum completed practice score shown separately.
- Admin school-wide and teacher-scoped CSV exports plus a print-ready report that can be saved as PDF; deployment, backups and release checks.

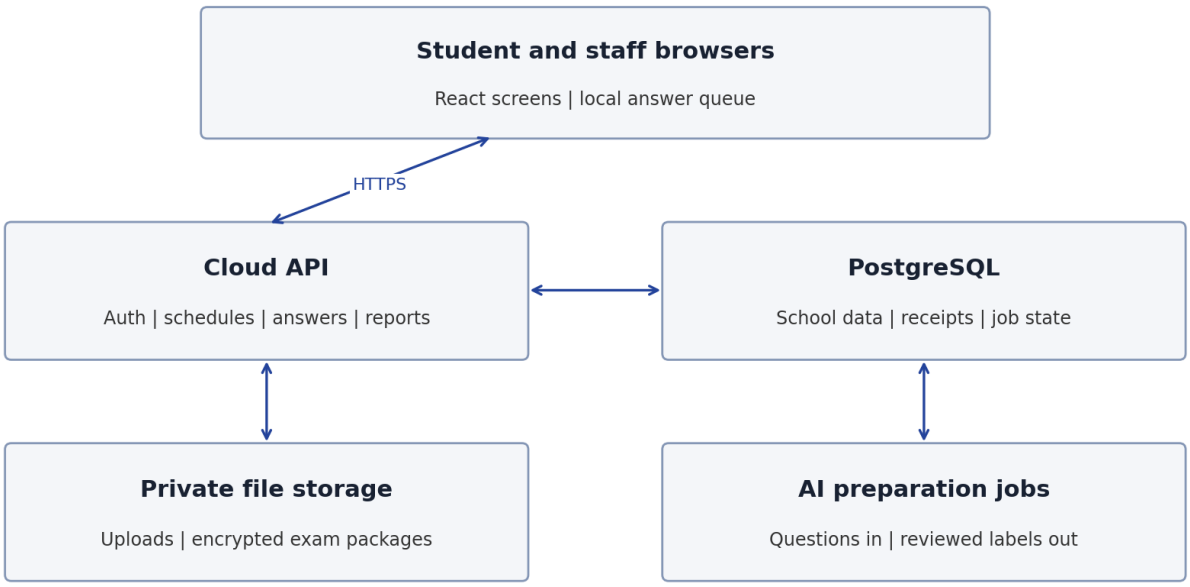
2.4 Scope boundaries that make the date possible

Use simple forms and one exam detail page with tabs. Defer essays, free-text grading, multi-select questions, scanned PDFs/OCR, unrestricted Word layouts, a visual question editor, mathematical equation conversion, automatic billing and notification campaigns. Unicode text is supported; rich images and diagrams inside question files follow after the first release. These are format and feature boundaries, not learner or exam-attempt quotas.

3. System architecture

3.1 Deployment shape

Use one React application, one modular Node API and one PostgreSQL database. Store school_id on school-owned records. Keep imports and generated exam packages private. The AI service runs while staff prepare questions, never in the student's official exam path.



3.2 Low Internet behavior

Students need Internet for login and downloading during preparation, a small unlock request at the common start, and the final confirmed upload. Once unlocked in an open tab, answering and navigation continue through Internet interruptions. School Wi-Fi must still have Internet at these connection points. No school PC runs a local server.

3.3 Environments and isolation

Environment	Configuration
Developer	Vite plus local API and a test database. A mock is allowed for an unfinished screen, not for acceptance.
Shared staging	Deployed frontend and API with seeded schools, teachers and students. Use generated test identities.
MVP release	The tested artifact, production secrets and separate database/storage. Mocks disabled. Actual school records require the agreed data-handling arrangement.

Keep one deployment for the initial schools. More schools do not require separate codebases or automated per-school servers. Expand capacity after measuring combined exam schedules and request load.

4. Technology stack and version policy

4.1 Keep the stack small

Area	MVP choice	Reason and implementation rule
Browser	React, TypeScript, Vite	Reuse the supplied project if healthy. Use the committed lockfile.
UI	Existing Tailwind and shadcn components	Keep the installed preset; do not redesign or regenerate the UI kit this week.
Routing and data	React Router, TanStack Query	Use existing compatible installs. One typed fetch/axios client; no second networking layer.
Forms	React Hook Form and Zod	Share request schemas where practical; the API still validates every request.
Exam storage	IndexedDB through idb	Use transactions for the answer snapshot and outbox. Keep any existing wrapper if it meets this contract.
API	NestJS modular monolith	Preserve the previously agreed architecture. A working equivalent backend need not be rewritten to change framework.
Database access	pg and versioned SQL migrations	A small explicit repository layer gives clear transactions and tenant filters. Reuse an existing ORM only if already working.
AI	One server-side provider adapter	Configure provider, model, key, timeout and budget. Choose a model on real question samples on Monday.
Charts and export	Existing chart library or simple SVG bars; CSV; print CSS	No paid reporting service or server-side browser rendering is required.
Tests	Vitest, API integration tests, Playwright, k6	Exercise real persistence, permissions, recovery and load. Use existing test tooling where available.

4.2 Runtime and package policy

Select a currently supported Node LTS runtime that the actual hosting account exposes and that all retained packages support. Pin it for local development and CI. Confirm package engine constraints and native-module compatibility on the host before writing more features. Exact versions belong in docs/VERSIONS.md, copied from the real install and lockfile, not from this PDF. [R4]

- Use npm ci in CI and deployment builds. Commit package-lock.json. Do not upgrade the frontend toolchain for novelty.
- Perform a clean install, build and basic security dependency review on Monday. Address blockers before integration; do not use force-upgrade commands blindly.
- No Redis, Kubernetes, microservices, Docker hosting requirement, vector database or local school network service is needed for this MVP.

4.3 Repository files

Keep AGENTS.md concise. Maintain docs/api-contract.md, docs/exam-engine.md, docs/VERSIONS.md and docs/deploy.md beside the code. Generate shared DTO types from one source where possible; do not hand-maintain conflicting copies. This guide is the initial specification, and tested changes must be reflected in those files.

5. Frontend architecture and components

5.1 Feature layout

```
src/app/              router, providers, role layouts
src/features/auth/    login, session and password reset
src/features/school/  accounts, roster, classes and subjects
src/features/exams/   import, AI review, approval and schedule
src/features/runner/  clock, reducer, local store and outbox
src/features/results/ scores, ranks, charts and exports
src/features/practice/ feedback, attempts and best score
src/components/       existing UI and shared components
src/lib/              API client, environment and formatting
tests/e2e/            real-backend user journeys
```

5.2 State and requests

State	Owner
Lists, exam metadata, results	TanStack Query. Key data by school, role, resource and filters; clear user caches on logout.
Logged-in user and role	A small auth context or existing store; the session credential stays in an HttpOnly cookie.
Official attempt	Reducer plus IndexedDB repository. React component state is never the only saved copy.
Forms and temporary UI	React Hook Form and local state. Server errors attach to fields where appropriate.

Normalize API errors to code, message, details and requestId. A 401 may require reauthentication, but must not erase pending exam work or trigger endless retries. Refetch results only after authorized release. Do not run a background query for every question.

5.3 Reusable components

Retain PageHeader, DataTable, FormField, ConfirmDialog, EmptyState and ErrorState. Add only the workflow pieces needed now: ImportPreview, AIReviewTable, SchedulePreview, CredentialIssueDialog, PreparationStatus, ExamTimer, SaveStatus, ReceiptPanel, TopicBars and PracticeFeedback. Prefer tabs and dialogs over many new pages.

5.4 Phone and recovery requirements

- Student screens must work at 360 CSS pixels wide with large tap targets, readable text and a collapsible question navigator. Test a real lower-memory Android phone.
- Use a service worker to cache the versioned application shell and required fonts. Never force an update while an attempt has pending work. Keep the active asset version until safe cleanup. [R3]
- Store every answer locally before showing Saved on this device. Show Last server save and Pending upload separately. A generic green Saved badge is insufficient.
- An offline reload must preserve stored work and show a recovery screen; resume editing only after the server clock and active attempt are revalidated. Open-tab continuation is the guaranteed MVP interruption path.

6. Frontend feature catalogue

6.1 Screens required for the working flow

Screen	Required controls and visible outcome
Login and account recovery	User ID and password; role redirect; admin-issued password reset; no student email dependency.
School setup	Classes, subjects, teachers, assigned classes/subjects; CSV student roster; generated credentials shown or exported once.
Teacher exam workspace	Title, subject, class, instructions, duration, marks, pass threshold, ordering rule; upload DOCX/XLSX; parse errors; AI status; review; Submit for approval.
Admin approval and schedule	View the complete exam, keys, labels and explanations; approve or return; edit preparation date/time; show calculated start and end; set result release and practice settings separately.
Student exam list	Assigned upcoming, preparing and completed exams. An exam link is navigation only and never grants eligibility.
Preparation and runner	Identity, download progress, storage check, countdown, question navigation, local/server save status, submit confirmation, receipt or pending recovery.
Admin exam dashboard	Counts of ready, started, confirmed submitted and recovery pending; last contact; official marks and ranks; score distribution, topic bars; exports.
Teacher results	Assigned exam/class results and CSV export; no approval, school-wide export or release controls.
Student Results tab	Exam title, official mark/percentage after release, status before release, separate best practice percentage and Practice action when allowed.
Practice runner	New practice session, first-answer scoring, correctness and approved explanation, progress, completed score and best-score update.
Recovery review	Admin sees late-upload reason, server receipt time, latest timely save and final snapshot; accept or reject with an audited reason.

6.2 Screen conventions

Every data screen has loading, empty, error-with-retry and populated states. Show date, EAT timezone and preparation/start labels together. A failed import points to the row and field. A queued AI task shows actual progress and a retry action. Disable buttons during a mutation, but still enforce duplicate-request handling on the server.

6.3 Credentials

Use a readable school prefix plus a unique sequence or random suffix for user IDs, and cryptographically random initial passwords. Do not use predictable passwords such as a birth date. Store only Argon2id password hashes with per-password salts [R8]. Staff privately issue the one-time credential list; later lost passwords are reset, not retrieved. Record who generated and reset accounts.

7. Question import and AI preparation

7.1 Two small import contracts

Teachers provide questions, options and correct answers, then enter exam-level parameters in the form. Topic labels and explanations are generated by the AI service. The MVP supports the provided templates rather than arbitrary Word documents. Both import routes produce exactly the same canonical question structure.

Format	Accepted structure
XLSX	One Questions worksheet, one row per item: external_id, type, question, option_a, option_b, option_c, option_d, answer, marks. MCQ uses answer A/B/C/D; true_false uses TRUE/FALSE and blank option cells.
DOCX	One simple table with the same column names and one row per question. No merged cells, nested tables, images or macros. A downloadable example removes layout guesswork.
Canonical item	Stable item ID; type mcq_single or true_false; stem; canonical option IDs; teacher key; positive integer marks. Metadata supplies subject, class and language.

7.2 Example teacher input

```
external_id: MATH-001
type: mcq_single
question: Solve 2x + 3 = 11.
option_a: 2    option_b: 4    option_c: 5    option_d: 7
answer: B
marks: 2
```

The form also requires exam name, instructions, duration in minutes, eligible class, subject and pass threshold. Display the calculated total marks. Save a draft during preparation so a failed AI request does not discard a teacher’s upload.

7.3 Validation and file handling

- Validate filename-independent type, ZIP expansion size, row count, missing fields, duplicate external IDs, positive marks and valid answer choices. Reject unsupported content with a useful error; never silently drop it.
- Use bounded parser libraries; disable external relationships, macros and formula execution. XLSX cells are data, not executable expressions. Do not render untrusted Word HTML.
- Initial technical safeguards: 5 MB per template and 200 questions per import, configurable after profiling. These protect parsing; they are not annual allowances or school license limits.
- Keep source uploads outside the public directory; use random private paths and authorized access. Retain the source hash and diagnostics. A preview must match the imported count, wording, options, keys and marks.

7.4 Fast implementation route

Build the XLSX parser and canonical validator first, then map DOCX table cells into that same validator. Use a maintained XLSX parser and a DOCX ZIP/XML reader already supported in the backend toolchain. Do not add OCR, general layout reconstruction or an LLM-based answer-key detector this week.

7. Question import and AI preparation continued

7.5 AI input and output

The backend sends question text, options, the teacher's answer, subject, grade, language and available topic names to one configured API model. Do not send student names, credentials, answer histories or result records. Treat uploaded text as data rather than instructions to the model.

```
{
  "itemId": "MATH-001",
  "primaryTopic": "Linear equations",
  "secondaryTopics": ["Algebra"],
  "explanation": "Subtract 3 to get 2x = 8, then divide by 2.",
  "keyConsistent": true,
  "reviewFlags": []
}
```

Validate the JSON schema and references to the supplied answer. Normalize topic names within the subject. If no topic list exists, AI may suggest one for staff review; teachers do not have to label every item manually. Give each item one approved primary topic so analysis denominators remain consistent.

7.6 Review and publication rules

1. Create an import job and draft exam. Persist parsed teacher questions and keys before starting AI work.

2. Generate topic labels and explanations in bounded background jobs. Record provider, model, prompt version, input hash, token usage, state and failure reason.

3. The teacher checks a side-by-side preview of question, supplied key, topic and explanation, then submits for approval. A conflict with the key is flagged; AI cannot change the key.

4. The school administrator approves the reviewed version or returns it with comments. Approval freezes the questions, keys, marks, primary topics and explanations.

5. Build the student package from a separate allowlisted schema that excludes keys and explanations. Scheduling requires approved content and a ready package.

7.7 Cost, latency and failure handling

Concern	MVP handling
Slow provider	Run a small number of jobs at a time, outside exam delivery requests. Start with two concurrent AI calls, a 30-second call timeout and at most two retries; tune from Monday's sample.
Process restarts	Persist jobs and lease expiry in PostgreSQL. A provider-supported cron invocation reclaims expired jobs and resumes unfinished items.
Repeated content	Cache within the school by question/key hash, model, prompt, language and taxonomy revision. Reuse approved explanations during every practice session.
Unavailable or wrong AI output	Keep the draft and diagnostic. Retry or let staff correct the draft; do not invent an explanation or silently approve incomplete content.
Budget and privacy	Track usage per school and enforce an operator preparation budget. Never send an API key to the browser; review provider retention settings.

The Saturday acceptance run must demonstrate a real API-generated label and explanation. Seeded text is useful for tests but does not prove the AI integration works.

8. Exam engine and submission protocol

8.1 Fixed schedule and preparation

```
starts_at = preparation_visible_at + 10 minutes
ends_at = starts_at + duration_minutes
student_deadline = ends_at + approved_extra_time
final_accept_until = student_deadline + 5 seconds
```

The school administrator chooses preparation_visible_at. Display the calculated start and end immediately, including the date and EAT timezone. Store UTC timestamptz. Extra time is an explicit per-student accommodation fixed before preparation. Late joining does not create a fresh full-duration timer.

Before preparation opens, show only the schedule. During the ten-minute preparation window, create or recover the student's unique attempt and download its encrypted question package and application assets. Mark Ready only after the checksum, asset download and a local storage write check succeed.

8.2 Unlock at the common start

Encrypt the question payload with AES-GCM, using a fresh random key and unique nonce for each encrypted package. Keep the key on the server until starts_at. The unlock request checks the authenticated student, school, roster, approval, package version, active device generation and server time before returning the key and deadline. The official package never contains correct answers or explanations. [R2]

A brief connection is required at the actual start. Encrypting preloaded questions is useful only while the key is withheld; sending the key ten minutes early and hiding a Start button would expose the paper. After unlock, keep navigation and answer changes independent of the network.

8.3 Runner state and local persistence

State	Meaning and next action
Scheduled / preparing	Wait for preparation or complete downloads. Questions are still locked.
Ready / unlocking	Package is complete; request the key at the fixed start. Retry transient network errors.
Active	Answer and navigate; persist the local snapshot and outbox in one IndexedDB transaction.
Submitting / pending upload	Freeze a complete final snapshot. Keep it until the server confirms receipt.
Submitted	The exact final snapshot has a durable official receipt. Editing is disabled.
Recovery pending	The final snapshot arrived too late or has a disputed device/version. Staff review is required.

Use a server-time offset and monotonic elapsed time in the open page; never extend the deadline because a device clock changes. Recompute remaining time after tab throttling. Allow one active tab per attempt; a second tab opens a read-only warning. On reload, recover the snapshot first and revalidate time online before resuming. Use IndexedDB transactions for local state. [R1]

8.4 Connected saves

Save locally on every change. When connected, send changed answers in a compact batch about every 30 seconds with a randomized initial offset. Piggyback server time and last-contact status. Back off failed sends and offer Retry now. Do not poll every student every three seconds; the admin monitor can refresh every 15 seconds.

8. Exam engine and submission protocol continued

8.5 Sequence and duplicate rules

Each local edit gets a monotonically increasing clientSeq within the attempt and active device generation. The API acknowledges specific item revisions. A newer revision replaces an older one; identical retries return the same acknowledgement. A reused revision with different content returns a conflict. Never clear an outbox entry just because its request was sent.

8.6 One atomic final submission

1. At manual submit or the deadline, freeze all answers in one complete snapshot, including locally unsent answers. Assign one idempotency key and retain the exact payload.

2. The API authenticates, scopes and locks the attempt. Check for an existing receipt before timing checks: identical key/payload returns that receipt even after the deadline; conflicting reuse returns 409. Otherwise validate items, timing and device generation.

3. In one database transaction, store the final snapshot, deterministic scores, final sequence, payload digest and unique receipt. This transaction overrides older interim saves but cannot be changed by a later save request.

4. Return the receipt only after commit. If the response is lost, resending the same payload and key returns the same receipt. Different content with that key is a 409 conflict.

5. If another save, submit or device transfer races, the row lock and state checks produce one final outcome. The browser shows Submitted only for an official committed receipt.

8.7 Deadline and late-upload policy

The UI locks at student_deadline. Ordinary answer saves are rejected after that time. A final snapshot received in full by the API within the next five seconds is accepted as an on-time transport allowance; this allowance never increases the visible timer. Use the server's receipt timestamp, not a client timestamp. This small allowance tolerates network transit but cannot prove that a modified browser stopped editing at the bell.

After final_accept_until, the recovery route stores the frozen snapshot separately and returns a recovery reference, not an official submission receipt. The admin may accept it with an incident reason or reject it. Keep the last timely server save as separate evidence. The suggested automatic retry window is ten minutes; later recovery is staff-assisted. No browser can prove when disconnected answers were written.

8.8 Failure behavior

Situation	Required outcome
Internet fails after unlock	Continue in the open tab; show device-only saves and queue changes.
Final upload times out	Retain the frozen payload and retry idempotently; never discard it after five seconds.
Deadline passes without a final upload	Show Pending upload; later uploads follow the late-recovery policy.
Scheduler notices an expired attempt	Mark awaiting upload/review; do not fabricate a complete submission from missing answers.
Server says attempt is closed	Fetch its actual receipt/state. Closed alone does not mean the device snapshot was received.
Device is lost or storage is cleared	Only server-confirmed work is automatically recoverable. An approved transfer invalidates the old device generation.

Keep pending local work through login expiry and ordinary updates. Device storage is a recovery aid, not an independent backup. Do not depend on browser Background Sync or background timers running after the tab is closed.

9. Backend architecture and rules

9.1 Modules and boundaries

Module	Responsibilities
Auth and directory	Sessions, generated credentials, staff roles, classes, subjects and roster import.
Content	Template parsing, draft questions, AI jobs, review, approval and immutable exam versions.
Delivery	Eligibility, preparation packages, unlock, local-save acknowledgements, final receipts and recovery review.
Results	Deterministic marking, ranked official results, publication timing, topic metrics and scoped exports.
Practice	Authorized practice sessions, first-answer feedback, completion and best-score calculation.
Operations	Job runner, health, structured logs, backups and protected tenant provisioning.

9.2 Requests and jobs

Every request passes validation, authentication, school scope and role/assignment checks before business logic. Services own transactions; repositories receive the verified school context. Do not trust a school ID supplied in JSON. Private downloads and exports must pass the same checks as ordinary records.

Persist AI/import job state before returning a job ID. Use a provider-supported cron command or authenticated runner endpoint to claim a small batch with database leases. Recover expired leases after a crash. Keep job concurrency low during exams. The API checks schedules at request time, so a delayed cron job cannot release a paper or results early.

9.3 Exact grading and release rules

- MCQ and true/false use the teacher-approved canonical key. Full marks for a correct answer; zero for incorrect or unanswered. MVP marks are positive integers, with no negative or partial marking.
- An official result becomes visible only when results_approved_at is set, results_release_at has arrived, and unresolved recovery cases are either decided or explicitly excluded with a reason.
- Rank finalized valid official totals within the same exam sitting and class. Use competition ranking: 1, 2, 2, 4. Absent, invalidated and unresolved attempts are status rows, not zero-scoring ranked students.
- Topic performance equals earned marks on the primary-topic items divided by available marks on those items for included attempts. Count unanswered items in the denominator. Show item and student counts.
- Practice requires the admin switch, practice release time, official result release for the cohort, and closure of all related sittings using the same paper. It never changes official marks or ranks.

9.4 Repeat practice

For each practice session, score the first submitted answer to each item and then return correctness plus the approved explanation. Completing all items finalizes that session. Best practice percentage is the maximum over valid completed sessions for the same student and exam version. Abandoned sessions do not update it. Use an atomic upsert or transaction so simultaneous completions cannot lower the maximum. A corrected or invalidated score triggers recomputation.

10. Data model and integrity

10.1 Minimum schema for the MVP

Use UUID primary keys, UTC timestamptz and explicit constraints. JSONB is appropriate for a versioned question snapshot or a final answer map, not for replacing relational school ownership. Every school-owned table includes school_id; composite foreign keys bind child records to the same school.

Table or group	Essential fields and constraints
schools	id, name, slug, timezone, status. Default timezone Africa/Addis_Ababa.
users	id, school_id, login_id globally unique, display_name, role, password_hash, status. Optional email; protected MFA secret reference for admins.
auth_sessions	session token hash, user_id, school_id, expiry, mfa_verified_at, revoked_at; record credential issuance/reset in audit_events.
classes and subjects	school_id, name, academic_year or code; unique identifiers within school.
enrollments and teacher_assignments	Student/class/year; teacher/class/subject. Use these for eligibility and teacher export scope.
topics	school_id, subject_id, normalized_name; unique per subject after staff review.
exams	school_id, owner_id, title, subject_id, class_id, instructions, duration_minutes, pass_percent, state. One class/sitting per exam in this MVP.
exam_versions and exam_items	exam_id, version, approved_by/at, package hash; item_id, position, type, stem, options, private key, marks, primary_topic_id, explanation. Frozen on approval.
exam_sessions and session_roster	exam_version_id, preparation_visible_at, starts_at, ends_at, schedule_revision; eligible student_id, extra_time, admission status. Snapshot roster before preparation.
exam_packages	exam_version_id, private ciphertext path, hash, nonce, protected key reference, readiness state. Do not store a public plaintext paper.
import_jobs and ai_jobs	source hash/path, status, progress, diagnostics; item hash, model/prompt, output, attempts, cost, lease_until, retry_at.

10.2 Tenant keys and indexes

Give referenced school-owned parents UNIQUE(school_id, id). Child keys include school_id so a reference cannot cross schools. Index users(login_id), enrollments(school_id, class_id), exams(school_id, owner_id, state), exam_sessions(school_id, starts_at), jobs(status, retry_at), and the attempt/result lookups on the next page. Make child school_id non-null.

10.3 Why approved versions exist

An edited draft must not mutate a live paper or practice explanation. Approval pins one version; material edits require a new version and approval. Use the same paper for the class. Optional question/option shuffle is generated once at attempt creation and stored against canonical IDs; order-dependent options stay fixed. Freeze roster and schedule before preparation; emergency reschedules require an audit reason.

10. Data model and integrity continued

Table	Essential fields and constraints
attempts	school_id, session_id, student_id, device_generation, deadline_at, state, last_seen_at, question_order, option_orders. Unique per school/session/student.
attempt_answers	school_id, attempt_id, item_id, value, client_seq, payload_hash, saved_at. Unique per attempt/item; reject same revision with different data.
submissions	school_id, attempt_id, final_seq, snapshot JSONB, digest, idempotency_key, receipt_id, received_at, accepted_at. UNIQUE(attempt_id); unique receipt and request key.
recovery_uploads	school_id, attempt_id, immutable snapshot/digest, received_at, reason, decision, decided_by/at. Separate from an accepted submission.
official_results	school_id, attempt_id, revision, score, max_score, percent, status, source_submission_id or recovery_id. Preserve prior revisions.
result_publications	school_id, session_id, revision, results_approved_at, results_release_at, excluded_attempt_ids, approved_by. API serves only the authorized revision.
practice_access	school_id, exam_version_id, enabled, release_at, eligible cohort; record which related sittings must be closed.
practice_attempts and practice_answers	student_id, exam_version_id, state, score, max_score, completed_at; unique first answer per practice attempt/item.
practice_summaries	school_id, student_id, exam_version_id, best_percent, best_attempt_id, completed_count. Unique on school/student/version.
audit_events	school_id, actor_id, action, resource_id, reason, server_at. Never log passwords, secret keys or complete answer payloads.

10.4 Transaction boundaries

- Answer save, final submission and device transfer lock the same attempt row. Reject stale device generations and ordinary writes after finalization.
- The final snapshot, official objective score and receipt commit together. Receipt reads remain idempotent even if the original response was lost.
- Approval, result publication and recovery decisions record actor and reason. A late acceptance after publication creates a revised result/publication; it does not overwrite history silently.
- Practice completion and best-score update are one transaction. For equal scores retain a documented tie choice, such as the earliest completed attempt.

10.5 Storage and retention

Index attempts(school_id, session_id, state), official_results(school_id, attempt_id, revision), submissions(attempt_id), and practice_attempts(school_id, student_id, exam_version_id, completed_at). Join results through the indexed attempts table for sitting reports. Generate UUIDs in the application so a missing database extension does not block deployment.

Use a short documented retention period for raw operational logs; retain academic records under the school agreement. Do not purge unreceived device work automatically. Keep finalized local data only for the agreed recovery period, then remove exam keys and student-specific browser data on safe cleanup. Export and offboarding procedures must be documented.

11. Security and school isolation

11.1 Authentication that fits the week

For a new backend, use opaque random sessions in HttpOnly, Secure, SameSite=Lax cookies, with hashed session tokens in PostgreSQL. Rotate the session at login and password change. Enforce expiry and revocation server-side. This avoids adding a fresh access/refresh JWT system solely for the MVP. If a secure token implementation is already working, retain it and document the same recovery behavior. [R6]

Protect mutations with exact Origin validation and a CSRF token. For app and API subdomains, use an exact CORS allowlist and credentialed requests. Use narrow login/session lookup functions to resolve identity before setting tenant context; never give normal handlers a global database bypass. Rate-limit by account plus a generous school-aware IP policy: hundreds of students may share one public IP.

11.2 School and role enforcement

- Resolve school and role from the verified session; recheck active membership. Query resources using both ID and school_id. Test all private endpoints using two seeded schools.
- Restrict teachers to assigned classes/subjects and owned or explicitly shared exams. Only school admins approve content, schedule release, enable practice and decide late recovery.
- Use PostgreSQL row-level security on tenant tables with transaction-local school context, a restricted runtime role and separate migration credentials. FORCE ROW LEVEL SECURITY prevents ordinary owner bypass; a superuser/BYPASSRLS role is unsuitable. Verify host privileges on Monday. [R5]
- If the account cannot support this configuration, use a compatible database before live multi-school use; do not quietly advertise database isolation that is absent.

11.3 Browser and data controls

Risk	Required control
Early paper or answer leakage	Encrypted preload, server-held start key, separate official/practice response schemas, no public files.
Lost student work	Transactional local saving, retained final payload, idempotent receipt and audited late recovery.
Untrusted files and AI output	Bounded parsers, plain text rendering, schema validation, staff review and no tool execution by AI.
Stolen staff access	Strong credentials and admin MFA before live school data; short operator access and audit records.
Sensitive browser caches	Cache-Control private/no-store for credentials, keys, results and feedback. Cache only explicitly allowed shell/package assets.
Cross-school exports	Authorize generation and download; escape spreadsheet-formula prefixes in exported text.

11.4 Examination conduct

Use identity checks, a single active official device, timing, optional fullscreen and low-volume focus events as supervision aids. A normal website cannot prevent screenshots, another phone or every app switch. Do not block accessibility shortcuts indiscriminately or auto-penalize a student for a focus event. Browser activity is evidence to review, not a verdict.

Confirm hosting, backup and AI data locations, school processing terms and retention before importing actual student records. HahuCloud advertises a UK data center; an Ethiopian reseller name alone does not establish local storage. Keep test environments on generated identities while this is resolved. [R7]

12. API contracts

12.1 Shared API conventions

Use /api/v1 as the logical prefix; map it through the hosting configuration. JSON dates are UTC ISO 8601. Validate request bodies and query filters; return requestId and stable error codes. Get endpoints have no enrollment side effects. Apply idempotency to account batches, approval, attempt creation, final submission and practice completion.

Endpoint group	Contract and access
POST /auth/login; POST /auth/mfa/verify; GET /auth/me; POST /auth/logout	loginId/password and session cookie. Admin MFA challenge grants no privileged access until verified. me returns user, school, role and CSRF token.
POST /students/import; POST /students/generate-credentials; POST /students/:id/reset-password	Admin only. CSV validation preview, batch confirmation and one-time credential issue. Existing account generation retries must not rotate passwords.
GET/POST /classes, /subjects, /teachers; assignments routes	Admin manages directory; teachers receive only their authorized teaching scope.
POST /exams; PATCH /exams/:id; POST /imports; GET /imports/:id	Teacher saves draft metadata, uploads template, polls progress and sees row diagnostics.
GET/PATCH /exams/:id/items; POST /imports/:id/ai-retry	Teacher reviews generated labels/explanations and fixes draft content. Require revision on writes.
POST /exams/:id/submit-for-approval; POST /exams/:id/approve or /return	Teacher submits; admin approves/returns. Freeze approved version and preserve approval history.
POST /exams/:id/sessions; PATCH /sessions/:id/schedule	Admin sets preparation time; API calculates start/end and snapshots roster. Reject active-sitting edits.
POST /sessions/:id/results-publication	Admin approves result revision and sets releaseAt; students cannot see it before due.
POST /exams/:id/practice-access	Admin sets enabled and releaseAt; enforce related-sitting and publication checks.
GET /sessions/:id/results; GET /sessions/:id/analytics; GET /exports/results.csv	Admin school-wide; teacher assigned scope. Filters include class/session and result revision.
GET /exports/report; GET /me/exams; GET /me/results	Authorized print-ready report; student own schedule and released results with separate best practice.

12.2 Scheduling request example

```
POST /api/v1/exams/{id}/sessions
{ "preparationVisibleAt": "2026-10-17T06:00:00Z",
  "resultsReleaseAt": "2026-10-17T11:00:00Z" }
Response for a 60-minute exam:
{ "startsAt": "2026-10-17T06:10:00Z",
  "endsAt": "2026-10-17T07:10:00Z", "scheduleRevision": 1 }
```

The UI displays 09:00 preparation, 09:10 start and 10:10 end in EAT. The student sees the official result at 14:00 EAT only if the administrator has approved the publication.

12. API contracts continued

12.3 Delivery and practice endpoints

Endpoint	Important response or rule
POST /sessions/:id/attempts	Create or return the student's unique attempt; do not create a fresh deadline on retry.
GET /attempts/:id/package; POST /attempts/:id/unlock	Encrypted payload available from preparation until the student deadline. Unlock checks server time and returns key, deadline, serverNow and deviceGeneration.
POST /attempts/:id/answers	Batch changed item revisions. Acknowledge accepted item/sequence pairs; do not silently accept conflicting data.
POST /attempts/:id/submit; GET /attempts/:id/receipt	Complete snapshot and idempotency key. Official receipt only after commit; receipt endpoint returns actual state if none exists.
POST /attempts/:id/recovery; POST /recovery/:id/decision	Preserve late snapshot; admin accepts/rejects with reason, creating official revision if accepted.
POST /attempts/:id/device-transfer	Admin authorizes new generation; recover only server-confirmed work automatically.
POST /exams/:id/practice-attempts; POST /practice-attempts/:id/answers	Check practice permission; create separate session; first answer returns correctness and approved explanation.
POST /practice-attempts/:id/complete	Idempotent final score plus atomic bestPracticePercent; incomplete attempts cannot finalize.
GET /health; GET /ready	Liveness and database/storage readiness. No credentials or detailed internals in public responses.

12.4 Final submission contract

```
POST /api/v1/attempts/{id}/submit
Idempotency-Key: <random request UUID>
{ "examVersion": 1, "scheduleRevision": 1,
  "deviceGeneration": 1, "finalSeq": 42,
  "answers": [{ "itemId": "q1", "value": "option-b" }] }

Committed response:
{ "status": "submitted", "receiptId": "receipt-uuid",
  "acceptedAt": "2026-10-17T07:10:02Z",
  "snapshotDigest": "server-computed SHA-256" }
```

Include every item, using null for unanswered items. Reject finalSeq below the highest accepted revision for this device generation. Server serialization determines the canonical digest; a client digest is only a comparison aid. A racing save cannot change a committed snapshot. Late recovery returns recovery_pending with a different reference type.

12.5 Errors the UI must handle

Use 401 AUTH_REQUIRED, 403 FORBIDDEN, 404 NOT_FOUND, 409 REVISION_CONFLICT / DEVICE_REPLACED / ALREADY_FINALIZED, 413 PAYLOAD_TOO_LARGE, 422 VALIDATION_ERROR and 429 RATE_LIMITED. Domain codes include NOT_PREPARING, NOT_STARTED, AI_REVIEW_REQUIRED, RESULTS_NOT_RELEASED, PRACTICE_NOT_RELEASED and RECOVERY_REQUIRED. Retry network/temporary errors with backoff; preserve work and surface permanent conflicts for staff action.

13. Delivery plan to Saturday

13.1 Planning assumption and critical path

Target Saturday 17 October 2026, with the integrated release candidate ready on Friday. Budget roughly 80 to 100 focused engineering hours across two developers or two actively owned implementation tracks, plus owner acceptance time. Existing scaffolding and familiar deployment are assumed. One person working only 3 to 4 hours daily cannot reliably deliver this whole scope from scratch by Saturday.

Assign one owner to API/database/deployment and one to frontend/exam storage; the product owner reviews academic rules and real-device behavior daily. Coding assistants can accelerate bounded tasks, but their output still needs review. The critical path is actual hosting and login, then one real exam submission, then recovery, approval/import/AI, and finally publication/practice/reporting.

13.2 Daily delivery checkpoints

Date	Backend and integration	Frontend and acceptance
Mon 12 Oct evening	Verify checkout, host runtime, database privileges and startup/restart. Create migrations, two test schools and real login. Deploy health endpoint.	Reuse UI kit; freeze DTOs and routes; 360px login/runner skeleton. Confirm template, exam rules, AI key and sample questions.
Tue 13 Oct	Build unique attempt, fixed schedule, encrypted package/unlock, answer save and atomic final receipt. Use a seeded objective paper.	Build preparation, timer, local queue and receipt UI against real API. Demonstrate start, an Internet interruption and successful submit on phone and PC.
Wed 14 Oct	Roster/credential issue, template parsers, AI jobs, review/approval and scheduling. Add recovery decision route.	Connect import preview, AI review, admin approval, schedule preview and recovery screen. Teacher-created exam reaches the runner without SQL edits.
Thu 15 Oct	Official marking/publication, ranks, topic aggregates, practice state and best score; scoped CSV and report endpoints.	Results tab, admin/teacher results, topic bars, distribution chart, practice feedback, best score and exports. Complete the entire school workflow.
Fri 16 Oct	Run real API isolation, duplicate/race, recovery, backup/restore and 500-student load scenarios. Tune host settings and fix failures.	Test phones, poor connectivity, refresh/relogin, timezones and print reports. Freeze features; rerun affected acceptance cases.
Sat 17 Oct	Restore smoke check, deploy the tested build, verify receipts and readiness. Keep rollback artifact.	Owner acceptance with a newly imported paper; staff walkthrough, known limitations and operating guide. Reserve the day for fixes, not new features.

13.3 Stop rework early

By Tuesday evening, a phone must produce a real database-backed receipt through the deployed API. If that is missing, stop new screens and fix the exam path. By Thursday evening, a teacher-created exam must run through admin approval, release and practice. Protect Friday’s verification time; do not hide missing backend work behind a mock.

13. Delivery plan to Saturday continued

13.4 Definition of a fully working MVP

Acceptance case	Pass condition
School isolation and roles	School A cannot access School B's students, exams, keys, results or exports. Teachers cannot approve or release.
Credentials	Unique generated IDs; passwords and admin MFA work. Reset revokes old sessions without destroying stored pending answers.
Import and AI	Real DOCX and XLSX templates yield correct items/keys. A real model generates usable topics/explanations; bad output stays in review.
Scheduling	Visibility plus ten minutes equals start; all students share the base deadline; questions cannot be read before unlock.
Interruption and receipt	Disconnect after start, answer, reconnect and submit: exact final answers and one receipt. Lost response and repeated submit do not duplicate data.
Late recovery and races	Deadline offline creates a pending case, not a fake submission. Concurrent save/submit/transfer has one valid audited outcome.
Official results	Correct marks and 1,2,2,4 rank ties; scores hidden before approved release time; excluded recovery cases labeled.
Practice	Submitted answers return correctness/explanation. Completed scores 60%, 80%, 70% display best 80%; official result is unchanged.
Analysis and exports	Topic denominators, result rows and CSV/report output reconcile. Teacher export cannot cross assigned scope.
Operations	Build has mocks disabled; HTTPS works; secrets remain server-side; restore succeeds; rollback and restart are documented.

13.5 Capacity acceptance

Use 500 distinct authenticated students and realistic question packages on the selected host. Exercise the ten-minute preload, synchronized unlock, staggered saves, final submission wave and duplicate retries. Measure latency, error rate, resource throttling, accepted receipt count and database reconciliation. Test the school network separately; an API load test does not simulate 500 phones on one access point.

Planning targets: p95 ordinary saves below two seconds, final receipt below three seconds on the test network, and zero lost accepted submissions or duplicate receipts. These are targets to measure, not an SLA. If the selected host cannot pass, the functionality may be complete while a 500-student deployment remains unapproved; fix the bottleneck or change hosting before that sitting.

13.6 Cut order if the schedule slips

Cut decorative charts, extra filters, role color themes, reusable question-bank management and automated deployment polish first. Keep CSV and basic topic bars. Preserve phone access, both template imports, AI review, admin authority, receipt/recovery correctness, results, practice, tenant isolation and acceptance checks. If these required behaviors are incomplete, report the missing scope and revised date; do not label the release fully working.

14. Deployment and operations

14.1 HahuCloud deployment path

Use the selected Enterprise Plus account as the starting deployment target. Its public plan lists four cores, 4 GB RAM, 250 GB NVMe, Node.js support and unlimited bandwidth. Those descriptions do not prove the account's database support, process limits or 500-user throughput. Verify PostgreSQL access, privileges, connection limits, cron interval, application restart and resource throttling directly. [R7]

1. Build the frontend and compiled API in CI using the verified runtime. Check the actual emitted startup file path; configure cPanel/Passenger to use it.

2. Deploy Vite static assets to the app site and the API through the supported Node app setup. A same-origin /api route is preferred when available; otherwise use explicit app/API subdomains and exact CORS/cookie configuration.

3. Put private uploads and packages outside public_html. Configure secrets through the host environment. Use FTPS, SFTP or the provider's supported deployment channel; do not assume shell access.

4. Apply backward-compatible database migrations, restart through the confirmed host mechanism, then test health, login, preparation and one final receipt.

5. Keep a versioned rollback artifact. Never deploy or force a service worker refresh during an active sitting. A documented manual upload/restart is acceptable this week.

14.2 Environment configuration

```
DATABASE_URL, SESSION_SECRET, APP_ORIGIN, CORS_ORIGIN
FILE_STORAGE_ROOT, EXAM_KEY_ENCRYPTION_SECRET
AI_PROVIDER, AI_MODEL, AI_API_KEY, AI_TIMEOUT_MS
AI_JOB_CONCURRENCY, AI_BUDGET_LIMIT, JOB_RUNNER_SECRET
LOG_LEVEL, NODE_ENV
Frontend: VITE_API_BASE_URL, VITE_USE_MOCKS=false
```

VITE variables are public build-time configuration. Never place a database credential, AI key or package-unlock secret in them. Record database/key backup handling and rotation in docs/deploy.md.

14.3 Workload arithmetic

Five hundred 2 MB packages downloaded over 600 seconds are about 1 GB total and 13.3 Mbps average download throughput before overhead. A 30-second save interval is about 16.7 requests/second if every student has changes. Five hundred final submissions over five seconds average 100 requests/second; also test the sharper common-start burst. The earlier 100 to 250 ongoing save requests/second estimate is not implied by this save policy.

14.4 Monitoring and backups

Watch API errors, p95 write latency, failed unlocks, pending recoveries, AI job backlog, CPU/RAM/I/O throttling and database pool wait. Use a small shared connection pool, initially around five connections per API process, and tune against the actual database limit. Five hundred students do not need 500 database connections.

Take an encrypted independent database/files backup and restore it before release. Confirm exactly what provider backups include and their interval. A submission receipt proves a committed record, not an independent backup; define the actual recovery point and retain browser evidence until recovery is settled.

15. Risks and release decisions

15.1 Risks that affect this week

Risk	Owner action and fallback
Too little implementation time	Product owner confirms two active tracks on Monday. If only limited solo time is available, reset the date or explicitly reduce scope; remove invented percentage chances of delivery.
Frontend scaffold differs from the PDF	Run install, build and CI checks on the actual checkout. Reuse working components; do not spend a day reproducing exact claimed versions.
Host cannot run the required API/database	Verify the deployed health/login slice on Monday. If PostgreSQL, runtime or privileges are unavailable, select a compatible cloud database/host immediately; keep the same codebase.
Shared hosting throttles the exam burst	Measure unlock and final-submit waves on Friday. Reduce background job load and optimize queries; upgrade measured bottlenecks.
AI quality or quota fails	Test real language/subject samples Monday. Persist drafts; review flags; cache reviewed output; change provider behind one adapter if necessary.
Unstructured Word content consumes the week	Ship a strict table template and useful validation. General document reconstruction stays in the roadmap.
Student loses Internet at the bell	Freeze locally, retry final upload, preserve late evidence and use admin recovery review. Never call missing work submitted.
Phone storage, power or network fails	Test actual phones and school Wi-Fi; check local writes in preparation; keep staff supervision and approved recovery procedures.
False progress from mock integration	Every daily checkpoint uses real API/database. Mocks may aid development but cannot satisfy release acceptance.
Data location or school processing unresolved	Resolve the live-data arrangement with the school and provider; use generated identities for testing. Do not assume backups share the same jurisdiction as hosting.

15.2 Release decisions

Release is based on evidence: the owner observes the complete workflow, the integrity checks pass, the deployed build matches the tested build, and the requested concurrency has been measured. A functional demonstration and a cleared 500-student school deployment are distinct outcomes. State exactly which one has been achieved.

15.3 What this deadline does not authorize

The date does not justify changing the teacher’s answer key with AI, accepting cross-school access, exposing the paper early, losing a student’s local queue, or reporting an unreceived snapshot as submitted. Keep these protections small and explicit instead of building a large security dashboard.

16. Working with coding assistants

16.1 Short implementation tasks

Use coding assistants for bounded modules and repetitive UI wiring. A developer owns the code and verifies the result. There is no need for a separate architect conversation before every routine command. Follow the repository's actual instructions and permission rules; never assume the supplied PDF describes the current machine or the owner's skill level.

Task	Deliverable and evidence
A Foundation	Deploy login, tenant context, migrations and a two-school seed. Evidence: API requests cannot cross schools.
B Exam delivery	Schedule, encrypted package, unlock, saves, submit and receipt. Evidence: real browser receives one correct durable receipt.
C Browser recovery	Reducer, local transaction, outbox and final snapshot. Evidence: interruption, retry, reload recovery and lost-response cases.
D Preparation	Roster, credentials, DOCX/XLSX validation, AI jobs and admin approval. Evidence: uploaded paper becomes an approved sitting.
E Learning and reporting	Released results, ranks, topic charts, exports, practice and maximum score. Evidence: the worked example in section 18 reconciles.
F Release	Real-device tests, load/restore evidence and deployment runbook. Evidence: versioned artifact, results and rollback instructions.

16.2 Task card template

```
Goal: one user-visible behavior
Owner and dependencies: who integrates it and what already exists
Contract: endpoints, schema changes and permission rules
Files: relevant modules; avoid unrelated rewrites
Acceptance: observable success and failure cases
Output: changed files, checks run, results and remaining blockers
```

16.3 Daily working loop

1. Agree the next small behavior and its request/response contract. Update the contract before incompatible code changes.

2. Implement on a short-lived branch. Include meaningful tests for integrity or permission changes, not tests that simply repeat a getter.

3. Run type checks, build and the affected tests; then use the real integrated screen. Record what actually passed.

4. Review and merge working slices at least daily. Deploy to shared staging so the next track can use them.

5. Raise concrete blockers immediately with the failing command/error, affected milestone and proposed fix. Stop expanding scope after Thursday.

16.4 Handover package

Deliver the repository, committed migrations and lockfile, example Word/Excel templates, seeded test accounts, environment variable names without secrets, deployment/restart/rollback instructions, receipt/recovery operating guide and acceptance results. Source code without a reproducible working deployment does not satisfy the Saturday milestone.

17. Roadmap after the MVP

17.1 After the objective-exam MVP

Priority	Addition	Why it follows this week
Next	Essay and configured short-answer marking	Requires rubric workflow, moderation and practice-score treatment beyond deterministic objective items.
Next	Richer Word input, images and mathematical content	Needs format preservation, asset encryption, preview and device testing. Avoid silent content corruption.
Next	Reusable question bank and additional export formats	Adds authoring convenience and polished reporting without changing the basic exam loop.
Next	Dedicated invigilator role and recovery tools	Adds scoped staff monitoring and more complete incident workflows.
Later	Multi-campus structures and advanced analytics	Introduce when real school structures and reporting needs are known; do not impose artificial learner quotas now.
Later	Billing, subscriptions and customer onboarding portal	Manual operator provisioning and finance handling are sufficient for the initial MVP.
Later	Managed database, separate workers or larger server	Upgrade from measured resource pressure, recovery requirements and simultaneous school schedules.
Optional	Native apps, camera proctoring, additional languages	Separate product decisions with device, privacy and operating costs. A full offline school server is not in the current direction.

17.2 What remains stable as the product grows

Keep the same tenant IDs, approved exam versions, official receipts, result revisions and practice history when infrastructure changes. The API remains the authority for time and permissions. Different hosting should not create per-school forks or a second interpretation of the marking rules.

17.3 Full product versus this deadline

Saturday’s target is the complete agreed flow for the constrained question formats and simple reports described here. The larger implementation blueprint remains useful as a roadmap, but its broader question types, polished export suite and longer delivery estimate do not define this week’s acceptance. Update business estimates separately when committing to the next release.

17.4 Later capacity planning

Measure combined schedules before supporting several schools sitting together. Five schools with 500 learners may produce 500, 1,500 or 2,500 concurrent exam takers depending on timetables. Subscription count, disk space and the word unlimited are not concurrency benchmarks. Preserve one product and add API, database or job capacity at the observed bottleneck.

18. Worked examination and glossary

18.1 Saturday acceptance example

XXX School creates a Mathematics exam with ten questions worth two marks each. The teacher uploads a template and answer keys; AI proposes primary topics and explanations. The teacher reviews the output and the administrator approves the frozen version. Preparation is set to 09:00 EAT on 17 October, so start is 09:10 and a 60-minute exam ends at 10:10.

1. A student signs in with a generated user ID/password and downloads the encrypted package at 09:00. The system checks local storage and shows Ready. Questions are unreadable before unlock.

2. At 09:10 the online unlock request succeeds. The student answers five questions, loses Internet, answers the rest and continues in the same open tab.

3. At 10:10 the UI freezes all ten answers. If the server receives the complete final request by 10:10:05, it commits the snapshot and returns one official receipt. A repeated request returns that same receipt.

4. A second student reconnects at 10:12. That upload is recovery_pending. The admin reviews and records a decision; no automatic Submitted claim is made for it.

5. After all counted results are settled, the admin approves publication for 14:00. Student scores are hidden before then. The dashboard shows marks, ranks and the topic chart, and authorized CSV rows match the results.

6. The admin enables practice after the paper is safe to release. Completed practice scores of 60%, 80% and 70% produce a best score of 80%, while the official exam result stays unchanged.

18.2 Analysis example

If five accepted students answer three Algebra questions worth two marks each, the available Algebra marks are 5 × 3 × 2 = 30. If they earn 18 marks, the topic result is 60%. Unanswered items still count toward the 30 available marks. Secondary topic labels do not double-count the denominator. Practice answers never enter this official chart.

18.3 Terms used in this guide

Term	Meaning
Tenant	One school’s isolated workspace and records.
Prepared package	Downloaded encrypted question content, without keys or explanations.
Unlock	A small authorized request at the common start that releases the decryption key.
Outbox	Device-stored answer updates waiting for server acknowledgement.
Receipt	Evidence that the server committed a particular final answer snapshot.
Idempotent request	Repeating the same request produces the same outcome, not a second submission.
Recovery pending	Late or disputed work preserved for a documented administrator decision.
Best practice score	Maximum valid completed practice percentage for one student and exam version.

19. Decisions and technical references

19.1 Resolve on Monday without redesigning the product

Decision to verify	Owner	Default or required evidence
Available engineering time	Product owner	Two active tracks, daily review, Friday validation time. Confirm against actual availability.
Existing repository state	Frontend and backend leads	Clean install/build; real versions; useful components; actual backend progress.
Host runtime and PostgreSQL	Backend lead	Successful deployed login, transaction and RLS test; known process/pool/cron/restart limits.
AI model and provider	Backend lead with teacher reviewer	One configured API passes sample English/Amharic questions, structured output and cost/retention review.
Academic rules	School admin and product owner	Objective-only MVP, common start, five-second final transport allowance, audited late recovery and competition ranks.
Template and example paper	Teacher reviewer	One DOCX and one XLSX sample pass import and preview without losing meaning.
Live-data arrangement	Nexora and school	Confirmed hosting/backup locations, processing terms, retention and authorized staff access.

19.2 Technical references

Reference document reviewed: Online-Exam-Platform-Documentation.pdf, version 1 dated 11 October 2026. Its numbered sections, blue subheadings, bordered tables and compact code blocks informed this layout. Earlier Nexora implementation version 2.0 and the project owner's requirements supply the product rules. Repository completion claims remain subject to the Monday checks.

R1 MDN IndexedDB API
https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

R2 MDN Web Crypto encryption
https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/encrypt

R3 MDN Service Worker API
https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

R4 NestJS deployment guidance
https://docs.nestjs.com/deployment

R5 PostgreSQL row security policies
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

R6 OWASP session management
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

R7 HahuCloud published plans and platform description
https://www.hahucloud.com/

R8 OWASP password storage
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Public references checked on 12 October 2026. All latency, resource, payload and effort numbers in this guide are planning assumptions until measured on the actual build and host.